KGCAS

co-Kreate your Genius

KG COLLEGE OF ARTS AND SCIENC

Autonomous Institution | Affiliated to Bharathiar Universi

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore – 641 035

Library book borrowing,return,and fine calculation system

NAME : MADHUMITA.C

REGISTER NUMBER: 25BCS231

COURSE NAME : B.Sc.CS

DEPARTMENT : B.Sc.CS-I-C

SUBJECT :JAVA PROGRAMMING

Introduction:

- Efficient library management is essential for tracking book movement and academic records.
- This project replaces manual book tracking with an automated system.
- Developed using Core Java and Swing GUI to provide a user-friendly interface.

Problem Statement:

- ***Manual Errors:*** Manual systems lead to inaccuracies in book tracking and fine calculations.
- ***Data Loss:*** Physical records are difficult to retrieve and prone to loss.
- ***Efficiency:*** There is a critical need for an automated way to count borrowing days and apply penalties.

Objectives of the Project:

- **Automate Operations:** To automate the core library functions of issuing and returning books.
- **Reduce Manual Effort:** To minimize the human workload involved in record-keeping.
- **Fine Calculation:** To implement a robust logic for Date Counting that automatically computes fines based on overdue days.
- **Data Accuracy**: To improve the precision of library data through digital storage.

Scope of the System:

- ***Educational Organizations:*** *Ideal for managing libraries in schools and colleges.*
- ***Training Centers:*** *Can be used by professional institutes to manage reading materials.*
- ***Small Libraries:*** *Scalable for local community library management.*

System Modules:

Module	Description
Security & Login	Authenticates the librarian using a **Username and Password** to ensure secure access to library records.
Book Management	Allows the librarian to input Book ID and Title to add new resources to the inventory via the **Add Book** button.
Borrowing Module	Records a transaction by linking a User ID to a Book ID and capturing the **Borrow Date** from the system clock.
Return & Fine Module	Automatically calculates **Late Days** and the **Fine** amount by comparing the return date with the recorded borrow date.
Reporting & Persistence	Uses the **View Issued** button to display history and the **Save** button to store data in text files using **File Handling**.

System Architecture/workflow:

Start

↓

User Registration / Login

(Librarian enters Username and Password via JOptionPane input dialog)

↓

Password Validation (System compares input with stored credentials using String comparison)

↓

Authentication Success / Failure (If Success: Opens Main GUI; If Failure: Shows Error Message & Exits)

↓

Access System Modules

Add Book: Register new book titles into the system.

Borrow Book: Input User/Book ID and capture the system Borrow Date.

Return Book: System performs Date Counting (Current Date - Borrow Date).

Fine Calculation: If days exceed the limit, a fine is automatically generated.

View Issued: Display a report of all active transactions from the file.

↓

Logout / Save (Click 'Save' to commit all changes to the text file using File Handling)

↓

End

Java Concepts Used:

- **Classes & Objects:** You defined multiple entity classes such as Person, User, Book, and Library to represent the system.
- **Inheritance:** You implemented a class hierarchy where User is a derived class that extends the base class Person using the extends keyword and calls the parent constructor via super().
- **Encapsulation:** In the Book class, variables like bookId and title are declared as private, and access is controlled through public Getters and Setters.
- **Polymorphism:** This is demonstrated through Method Overriding (inheriting from Person) and using specialized logic for different operations within the same system structure.
- **Exception Handling:** Your GUI buttons use try-catch blocks to catch Exception ex. For example, you use Integer.parseInt() inside a try block to handle invalid numeric inputs.

- **File Handling (Object Streams):** You implemented data persistence using ObjectOutputStream and ObjectInputStream to save and load the entire books ArrayList to a file named data.dat.
- **Java Swing (GUI)**: You built a graphical interface using JFrame, JButton, JTextField, and JTextArea. You also used JOptionPane for the initial Password Authentication dialog.
- **Event Handling:** You utilized Lambda Expressions (e -> { ... }) to handle action events for buttons like "Add Book", "Borrow", and "Return".
- **Date & Time API:** You used java.time.LocalDate and ChronoUnit.DAYS to perform the Date Counting logic required for calculating the fine when a book is overdue.
- **Collections Framework:** You used an ArrayList<Book> to dynamically store and manage the list of books in the library.

program:

```java
import javax.swing.*;

// Base class (Inheritance)
class Person implements Serializable {
    protected int id;
    protected String name;

    public Person(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

// Derived class
class User extends Person {
    public User(int id, String name) {
        super(id, name);
    }
}

// Book class (Encapsulation)
class Book implements Serializable {
    public LocalDate getBorrowDate() {
        return borrowDate;
    }

    public void setBorrower(User user) {
        borrower = user;
    }

    public User getBorrower() {
        return borrower;
    }
}

// Library class
class Library implements Serializable {
    private ArrayList<Book> books = new ArrayList<>();

    // Method Overloading (Polymorphism)
    public void addBook(int id, String title) {
        books.add(new Book(id, title));
    }

    public String borrowBook(int bookId, int userId, String n
        for (Book b : books) {
            if (b.getBookId() == bookId) {
                if (!b.isBorrowed()) {
                    User u = new User(userId, name);
                    b.setBorrowed(true);
                    b.setBorrowDate(LocalDate.now());
```

```java
public String returnBook(int id) {
    for (Book b : books) {
        if (b.getBookId() == id) {
            if (b.isBorrowed()) {

                LocalDate returnDate = LocalDate.now();
                long days = ChronoUnit.DAYS.between(
                        b.getBorrowDate(), returnDate);

                long lateDays = (days > 5) ? (days - 5) : 0;
                int fine = (int) lateDays * 5;

                User u = b.getBorrower();
                b.setBorrowed(false);

                return "Returned: " + b.getTitle() +
                        "\nUser: " + u.name +
                        "\nBorrow Date: " + b.getBorrowDate() +
                        "\nReturn Date: " + returnDate +
                        "\nLate Days: " + lateDays +
                        "\nFine: ₹" + fine;
            } else {
                return "Book Not Borrowed!";
            }
        }
    }
    return "Book Not Found!";
}

public String viewIssuedBooks() {
```

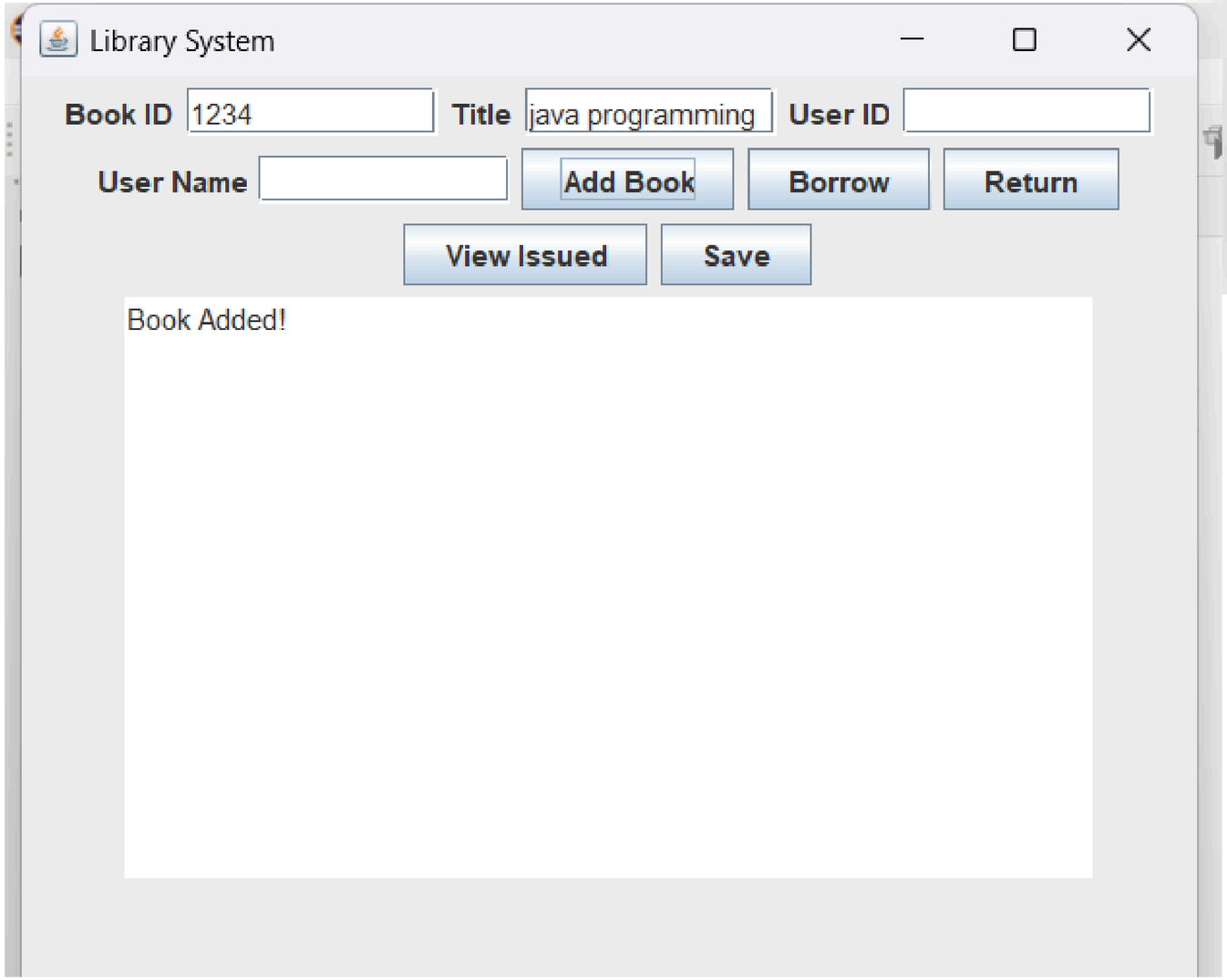
```java
        try {
            ObjectOutputStream oos = new ObjectOutputStre
                    new FileOutputStream("data.dat"));
            oos.writeObject(books);
            oos.close();
        } catch (Exception e) {
            System.out.println("Save Error");
        }
    }

    // File Handling (Input Stream)
    public void load() {
        try {
            ObjectInputStream ois = new ObjectInputStream
                    new FileInputStream("data.dat"));
            books = (ArrayList<Book>) ois.readObject();
            ois.close();
        } catch (Exception e) {
            System.out.println("No previous data");
        }
    }
}

// GUI Class
public class LibraryGUI extends JFrame {

    private Library lib = new Library();
    private JTextField bookId, title, userId, userName;
    private JTextArea output;

    public LibraryGUI() {
```

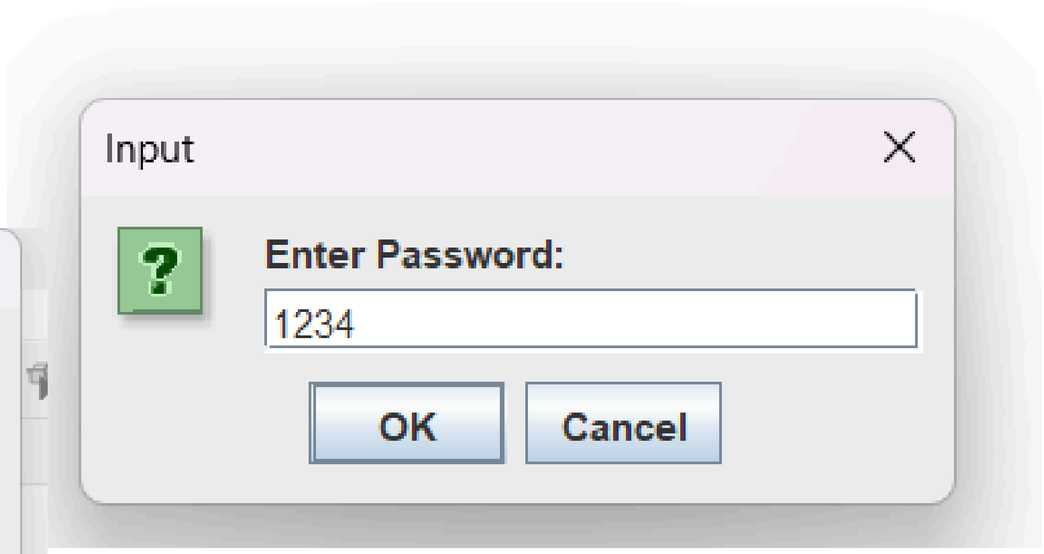
```java
        JOptionPane.showMessageDialog(this, "Invalid Login");
        System.exit(0);
    }

    setTitle("Library System");
    setSize(500, 500);
    setLayout(new FlowLayout());

    bookId = new JTextField(10);
    title = new JTextField(10);
    userId = new JTextField(10);
    userName = new JTextField(10);

    add(new JLabel("Book ID")); add(bookId);
    add(new JLabel("Title")); add(title);
    add(new JLabel("User ID")); add(userId);
    add(new JLabel("User Name")); add(userName);

    JButton addBtn = new JButton("Add Book");
    JButton borrowBtn = new JButton("Borrow");
    JButton returnBtn = new JButton("Return");
    JButton viewBtn = new JButton("View Issued");
    JButton saveBtn = new JButton("Save");

    add(addBtn); add(borrowBtn);
    add(returnBtn); add(viewBtn);
    add(saveBtn);

    output = new JTextArea(15, 40);
    add(output);
```

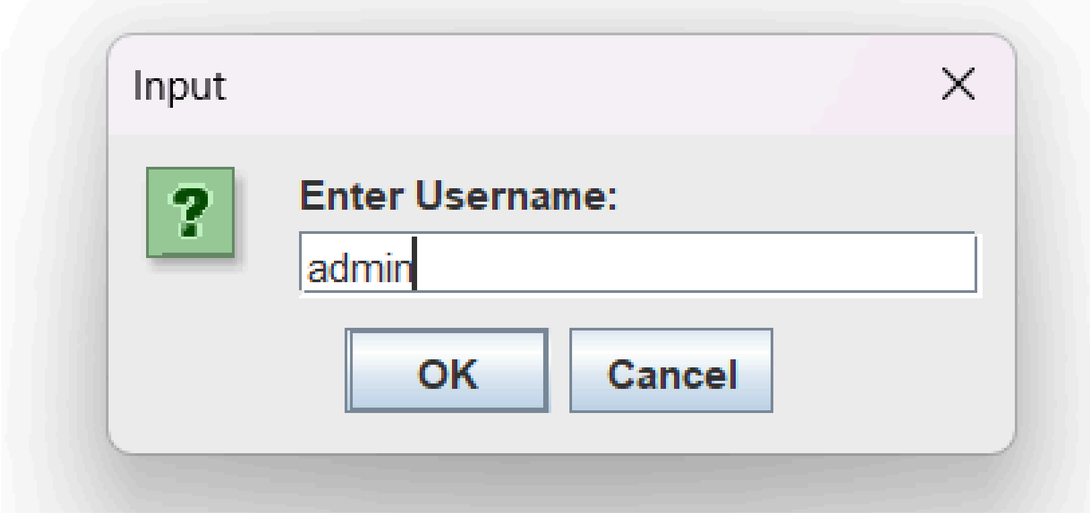
```java
// Actions with Exception Handling
addBtn.addActionListener(e -> {
    try {
        lib.addBook(
            Integer.parseInt(bookId.getText()),
            title.getText()
        );
        output.setText("Book Added!");
    } catch (Exception ex) {
        output.setText("Invalid Input!");
    }
});

borrowBtn.addActionListener(e -> {
    try {
        output.setText(
            lib.borrowBook(
                Integer.parseInt(bookId.getText()),
                Integer.parseInt(userId.getText()),
                userName.getText()
            )
        );
    } catch (Exception ex) {
        output.setText("Error!");
```

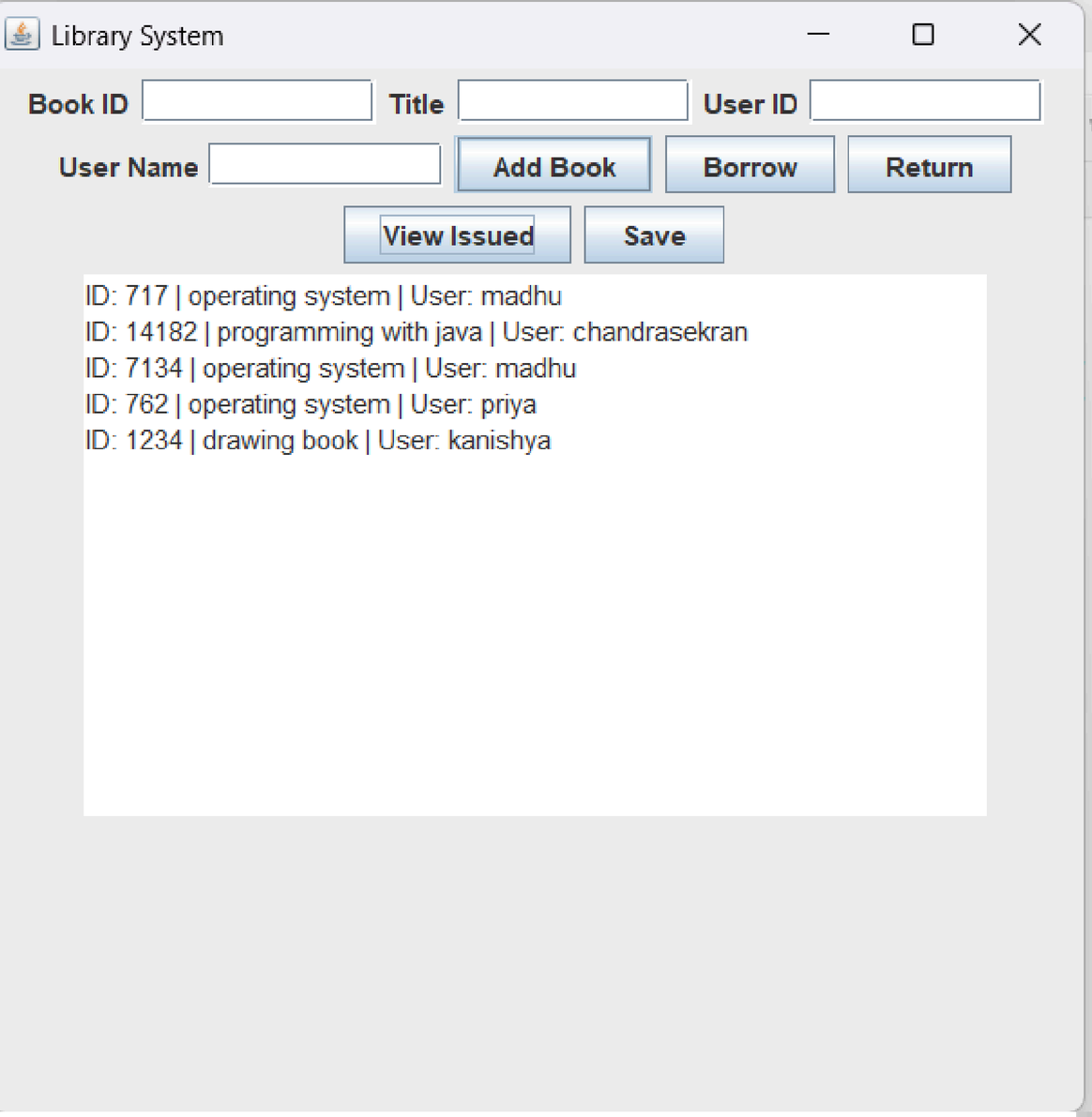
```java
                output.setText("Error!");
            }
        });

        returnBtn.addActionListener(e -> {
            try {
                output.setText(
                    lib.returnBook(
                        Integer.parseInt(bookId.getText())
                    )
                );
            } catch (Exception ex) {
                output.setText("Error!");
            }
        });

        viewBtn.addActionListener(e -> {
            output.setText(lib.viewIssuedBooks());
        });

        saveBtn.addActionListener(e -> {
            lib.save();
            output.setText("Data Saved!");
        });

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public static void main(String[] args) {
        new LibraryGUI();
    }
}
```

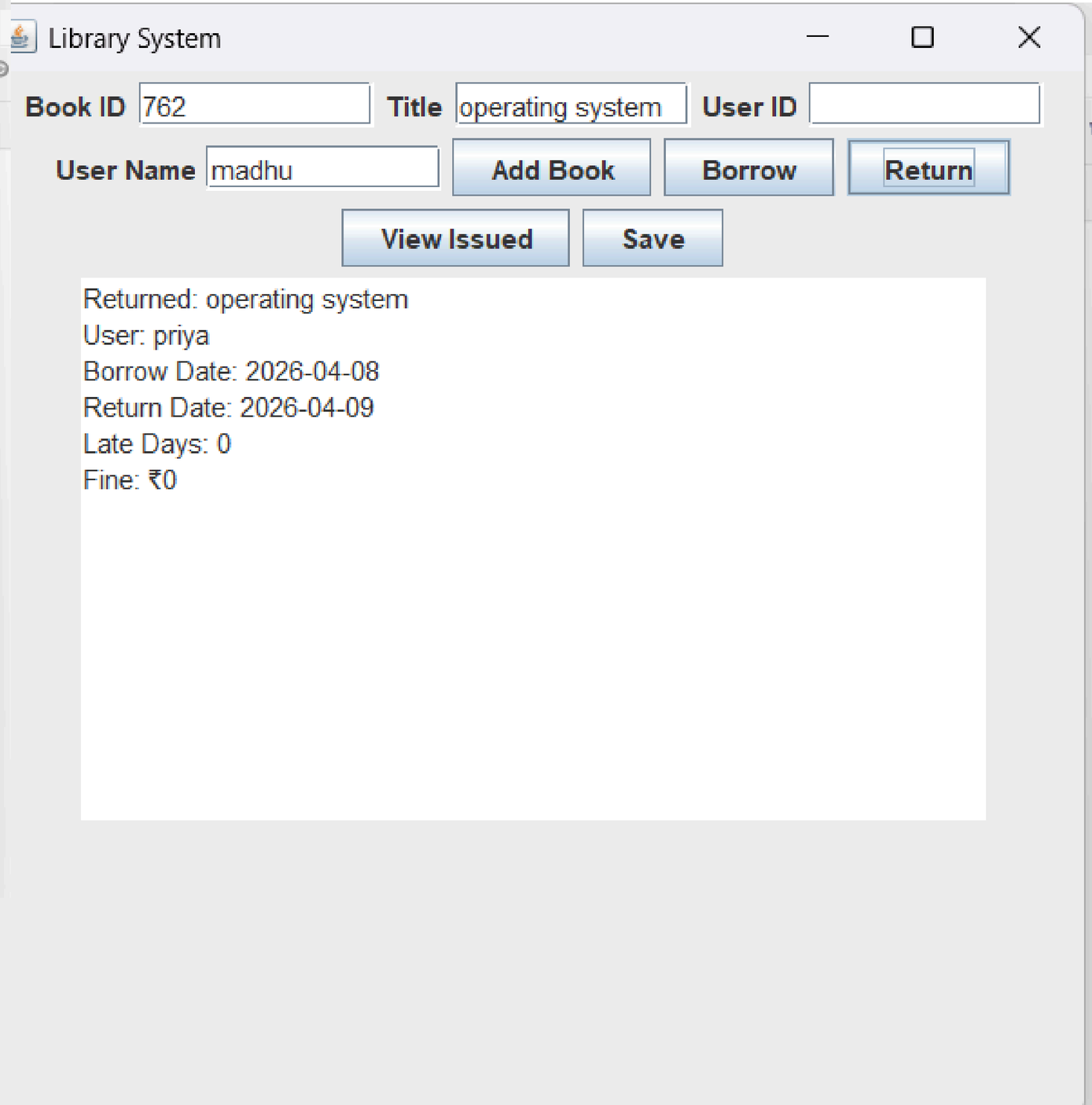
output:


Library System
Book ID
1234
Title
java programming
User ID
User Name
Add Book
Borrow
Return
View Issued
Save
Book Added!


Input
Enter Password:
1234
OK
Cancel


Input
Enter Username:
admin
OK
Cancel


Library System
Book ID
Title
User ID
User Name
Add Book
Borrow
Return
View Issued
Save
ID: 717 | operating system | User: madhu
ID: 14182 | programming with java | User: chandrasekran
ID: 7134 | operating system | User: madhu
ID: 762 | operating system | User: priya
ID: 1234 | drawing book | User: kanishya


Library System
Book ID
762
Title
operating system
User ID
User Name
madhu
Add Book
Borrow
Return
View Issued
Save
Returned: operating system
User: priya
Borrow Date: 2026-04-08
Return Date: 2026-04-09
Late Days: 0
Fine: ₹0

Demonstartion steps:

- enter the username and password.
- enter the bookID and book name.
- click add button to add the book. book added will be displayed in the text area.
- if any student want to borrow the book click borrow button the date student borrowed will be recorded.
- click save the datas will be saved for reloading when the library management open after closing for one day.
- The books issued were displayed when you click the view issued.
- the date of return will be noted , if the limited date overs fine amount and late days will be calculated by the computer and displayed on the text area .

Conclusion:

- The project successfully demonstrates the implementation of Core Java in a real-world scenario.
- Automation has significantly improved the efficiency of tracking books and managing overdue penalties.
- The use of OOP principles ensures the code is modular and easy to maintain.

Future Enhancements:

- Database Integration: Upgrading from text files to a MySQL database.
- User Roles: Creating a separate interface for students to check their own fines.
- Web-Based Interface: Developing a browser-based version of the system.